

VLSI-Based Digital Clock with Alarm Functionality

Abstract

This project presents the design and verification of a VLSI-Based Digital Clock with Alarm Functionality using Verilog HDL. The system tracks time in 24-hour format and generates an alarm signal whenever the current time matches a predefined alarm time. The design consists of a clock divider, cascaded counters, alarm comparator, and display interface logic. Functional verification was performed using EDA Playground and EPWave.

1. Introduction

Digital clocks are widely used in consumer electronics, embedded systems, industrial automation, and FPGA-based controllers. The purpose of this project is to design a reliable digital clock system using hardware description language and verify its operation through simulation.

The project demonstrates fundamental VLSI concepts including sequential logic, combinational logic, counter design, clock management, and digital verification.

2. Project Objective

The objectives of this project are:

- Design a digital clock using Verilog HDL
 - Implement a 24-hour timekeeping system
 - Generate alarm output upon time match
 - Verify functionality through simulation
 - Demonstrate modular RTL design methodology
-

3. System Architecture

The overall architecture consists of:

Clock Input

↓

Clock Divider

↓

Seconds Counter

↓

Minutes Counter

↓

Hours Counter

↓

Alarm Comparator

↓

Alarm Output

The clock divider generates periodic tick pulses that drive the cascaded counters. The alarm comparator continuously checks the current time against the programmed alarm time.

4. Clock Divider Design

The FPGA clock typically operates at high frequencies. Therefore, a clock divider is used to generate slower timing pulses.

Function:

- Receives high-frequency clock
- Counts clock cycles
- Produces periodic tick output

The generated tick signal acts as a one-second pulse for the clock system.

5. Counter Design

Seconds Counter

Range:

0-59

Behavior:

After reaching 59, the counter resets to 0 and increments the minute counter.

Minutes Counter

Range:

0-59

Behavior:

After reaching 59, the counter resets to 0 and increments the hour counter.

Hours Counter

Range:

0-23

Behavior:

After reaching 23, the counter resets to 0.

6. Alarm Comparator Logic

The alarm comparator compares:

Current Hour

Current Minute

with

Alarm Hour

Alarm Minute

Alarm condition:

alarm_enable = 1

AND

current_hour = alarm_hour

AND

current_minute = alarm_minute

When all conditions are true, alarm_out becomes HIGH.

7. RTL Modules

The project contains the following RTL modules:

1. clock_divider.v
2. digital_clock.v
3. alarm_comparator.v
4. seven_seg_decoder.v
5. top.v

The modular structure improves readability, verification, and scalability.

8. Testbench Design

The testbench performs:

- Clock generation
- Reset verification
- Counter verification
- Minute rollover testing
- Alarm trigger testing

The waveform results confirm the correct operation of the design.

9. Simulation Results

Simulation was performed using EDA Playground.

Observed Results:

- Tick generation verified

- Seconds counting verified
- Minute rollover verified
- Alarm comparator verified
- Alarm output generation verified

Waveforms clearly show the transition of counters and activation of alarm_out when alarm conditions are met.

10. Applications

This design can be used in:

- Digital clocks
 - Digital watches
 - Embedded systems
 - FPGA development platforms
 - Consumer electronics
 - Industrial automation systems
 - Timing controllers
-

11. Conclusion

A complete VLSI-Based Digital Clock with Alarm Functionality was successfully designed and verified using Verilog HDL. The implementation demonstrates essential digital design concepts such as clock division, counter design, comparator logic, and functional verification. Simulation results confirm the correctness of the design and its suitability for FPGA and VLSI applications.

References

1. Verilog HDL Documentation
2. FPGA Design Methodology
3. Digital Design and Computer Architecture
4. EDA Playground Documentation
5. EPWave User Guide